Experiment – 4 Validation of Forms using JavaScript

1. **Aim:** To design a form using HTML and CSS and apply client-side verification to it using JavaScript

2. **Objective:** To teach taking inputs form users and validate the inputs using HTML/CSS and JavaScript.

3. **Lab outcome:** After performing the experiment, the learner will be able to style web page using CSS and make them responsive.

4. **Prerequisites**: Basic knowledge of HTML and CSS

5. **Requirements:** A PC installed with Notepad software and a web browser

6. **Related theory:**

An HTML or a Web form helps collect user input. HTML forms can have different fields, such as text areas, text boxes, radio buttons, checkboxes, drop-down lists, file uploads, etc.

The collected data can be validated on the client browser using JavaScript. After validation of form data on the client-side, the user clicks on the Submit button in the form. After this, data is sent to the server for further processing.

The process of creating a form using HTML/CSS:

- HTML <*form*> element:
  The HTML <*form*> element is used to create an HTML form. It starts with the <*form*> tag and ends with the <*/form*> tag. We can add the input elements within the form tags for accepting user input.

```
<!-- File name: forms.html -->
<html>
<body>
    <!-- Creating Form -->
    <form>
      <!—Form elements are added here -->

    </form>

</body>
</html>
```

- HTML <*input*> element:

The HTML <*input*> element is used to create form fields and receive input from the user. Various input fields are used to take different information from the user. Using different Type attributes, we can display an <*input*> element in various ways.

Type	Description
`<input type="text">`	Displays a single-line text input field
`<input type="radio">`	Displays a radio button (for selecting one of many choices)
`<input type="checkbox">`	Displays a checkbox (for selecting zero or more of many choices)
`<input type="submit">`	Displays a submit button (for submitting the form)
`<input type="button">`	Displays a clickable button
`<input type="date">`	Defines a date picker. he appearance of the date picker input UI varies based on the browser and operating system. The value is normalized to the format *yyyy-mm-dd*.
`<input type="email">`	Displays a single-line text input field for entering email.

Some attributes of <*input*> element:

Attribute	Description
`value`	The input value attribute specifies an initial value for an input field
`size`	The input size attribute specifies the visible width, in characters, of an input field
`required`	The input required attribute specifies that an input field must be filled out before submitting the form.
`id`	The id attribute specifies a unique id for an HTML element. The value of the id attribute must be unique within the HTML document. The id attribute is used to point to a specific style declaration in a style sheet. It is also used by JavaScript to access and manipulate the element with the specific id.
`name`	The name attribute specifies the name of an <*input*> element. The name attribute is used to reference elements in a JavaScript, or to reference form data after a form is submitted. Only form elements with a name attribute will have their values passed when submitting a form.

Apart from <*input*> elements, some other related elements are required.

- The <*label*> Element

The <*label*> tag defines a label for many form elements. The <label> element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focuses on the input element.

The <l*abel*> element also helps users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the <*label*> element, it toggles the radio button/checkbox.

The for attribute of the <*label*> tag should be equal to the id attribute of the <input> element to bind them together.

Validation of inputs collected in the form using JavaScript:

Data validation: It is the process of ensuring that user input is clean, correct, and useful. Typical validation tasks are:

- Has the user filled in all required fields?
- Has the user entered a valid date?
- Has the user entered text in a numeric field?

Validation can be defined by many different methods, and deployed in many different ways. Server-side validation is performed by a web server, after input has been sent to the server. Client-side validation is performed by a web browser, before input is sent to a web server.

JavaScript: It is the language which is used for client-side validation. JavaScript is the programming language of the web. It can update and change both HTML and CSS. It can calculate, manipulate and validate data.

The *script* tag: In HTML, JavaScript code is inserted between <*script*> and <*/script*> tags. Any number of scripts can be placed in an HTML document. Scripts can be placed in the <*body*>, or in the <*head*> section of an HTML page, or in both.

Functions: JavaScript functions are used to validate inputs from various elements included in a form. A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses (). The code inside the function will execute when the function is invoked under one of the following conditions:

- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code
- Automatically (self-invoked)

When JavaScript reaches a return statement, the function will stop executing. If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement. Functions often compute a return value. The return value is "returned" back to the "caller".

Document Object Model (DOM): With the HTML DOM, JavaScript can access and change all the elements of an HTML document. When a web page is loaded, the browser creates a DOM of the page. The DOM defines:

- The HTML elements as objects
- The properties of all HTML elements
- The methods to access all HTML elements
- The events for all HTML elements

Following syntax is used to manipulate forms of the document.: `document.forms`

In JavaScript, you can submit the form using the '*submit*' input type and execute a particular function to handle the form data using the *onsubmit*() event handler.

7. Laboratory Exercise

A. Create a Registration Form using HTML and CSS. The form should contain fields for first name, last name, birth date, gender, email and phone number. Also include one submit button.

B. Use JavaScript to validate all the fields of the form.

8. Post Experimental Exercise:

A. Questions:

1. Compare static and dynamic website.
2. Explain the terms “domain name” and “hosting” with respect to websites.
3. List down 5 website-builder services which are free.

B. Results/Observations/Program output:

Submit the written lab report to the teacher in-charge. The lab report includes print out of the experiment-3 lab manual, hand written answers of post-experiment questions and conclusion and screenshots of the form created by you along with all the test cases for form validation. Upload folder of your *.html*, *.css* files on Google Classroom.

C. Conclusion:

Write what was performed in the experiment and which tools did you use for it. Write about how can you make the websites interactive and dynamic.

D. References:

[1] HTML 5 Black Book, 2nd Ed, DT Editorial services, 2016

[2] HTML & CSS: The Complete Reference, 5th Ed., Thomas A. Powell, Mc Graw Hill, 2010

[3] JavaScript: The Good Parts, I Ed., Douglas Crockford, O’Reilly, 2008